

FlashTeX Rendering Pipeline: Architecture Reference

FlashTeX Documentation Team

January 1, 1970

Abstract

This document is an internal reference describing the stages of the FlashTeX rendering pipeline, from source parsing through final PDF emission. It is intended as a map of the codebase for new contributors and exercises the cross-referencing machinery — table of contents, list of figures, list of tables, labels, and hyperlinks — that the pipeline itself must reproduce faithfully when compared against `pdflatex`.

Contents

1	Overview	2
2	Parsing	2
2.1	Tokenization	2
2.1.1	Category Codes	2
2.2	Macro Expansion	3
A	Note on Compatibility	3
3	Layout	3
3.1	Box and Glue Model	3
3.2	Page Breaking	3
4	Shaping	3
4.1	Font Selection	3
4.2	Kerning and Ligatures	3
5	Emission	3
6	Appendix: Glossary	4

List of Figures

1	High-level data flow through the rendering pipeline	2
---	---	---

List of Tables

1	The four pipeline stages	2
2	Selected default category codes	2

1 Overview

The pipeline is organized into four major stages, summarized in Table 1 on page 2. Each stage is described in its own section below, and Figure 1 on page 2 shows the data flow between them.

Stage	Input	Output
Parsing	<code>.tex</code> source	Token stream
Layout	Token stream	Box tree
Shaping	Box tree	Glyph runs
Emission	Glyph runs	PDF bytes

Table 1: The four pipeline stages

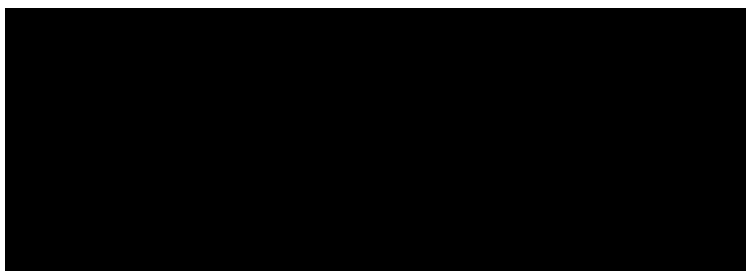


Figure 1: High-level data flow through the rendering pipeline

See also Section 6 for a glossary of terms used throughout this reference.¹

2 Parsing

2.1 Tokenization

The tokenizer converts raw source bytes into a stream of control sequences, character tokens, and category-code annotations, mirroring the behavior described in `tex.web`. Section 2.2 discusses how these tokens are subsequently expanded.

2.1.1 Category Codes

Every input character is assigned one of sixteen category codes before any macro expansion takes place. This table is user-configurable via `\catcode`, and the default assignment is summarized in Table 2.

Code	Meaning
0	Escape character
1	Begin group
2	End group
6	Parameter

Table 2: Selected default category codes

¹An expanded glossary, kept in sync with the codebase, is published at <https://example.org/flashtex/glossary>.

2.2 Macro Expansion

Macro expansion proceeds left to right, repeatedly replacing expandable tokens until only primitive and unexpandable tokens remain, as summarized by equation (1):

$$T_{n+1} = \text{expand}(T_n), \quad T_\infty = \lim_{n \rightarrow \infty} T_n. \quad (1)$$

We revisit this fixed-point view again in Section 3 when discussing how expansion interacts with the layout stage.

A Note on Compatibility

This unnumbered section is nonetheless listed in the table of contents via an explicit `\addcontentsline` call, which is a common pattern in reference documentation for marking important asides without disturbing the numbered section sequence.

3 Layout

3.1 Box and Glue Model

Layout consumes the expanded token stream and produces a tree of boxes connected by glue, following the classic TeX box-and-glue model. The badness of a line is computed from the total stretch or shrink required to reach the target width; see Table 1 for where this fits in the overall pipeline.

3.2 Page Breaking

Page breaking operates on the vertical list produced by paragraph breaking, choosing break points that minimize total badness across the document, consistent with the discussion in Section 2.1.

4 Shaping

4.1 Font Selection

Font selection resolves each glyph run to a concrete font file and glyph index, using the same NFSS conventions as L^AT_EX, cross-referenced back to the category-code table in Table 2.

4.2 Kerning and Ligatures

Kerning and ligature substitution are applied per shaping run before glyphs are positioned, a process that further refines the raw box widths computed in Section 3.

5 Emission

The final stage serializes the positioned glyph runs into PDF content streams. Further implementation notes are available at <https://example.org/flashtex/emission-notes>, and the full API reference is linked from [the API documentation](#).

6 Appendix: Glossary

- **Badness**: a numeric measure of how far a line or page deviates from its ideal width or height.
- **Glue**: a flexible space with a natural size, stretch, and shrink, as introduced in Section 3.
- **NFSS**: the New Font Selection Scheme used by L^AT_EX, referenced in Section 4.

This concludes the architecture reference; refer back to Figure 1 and Table 1 for the high-level summary.